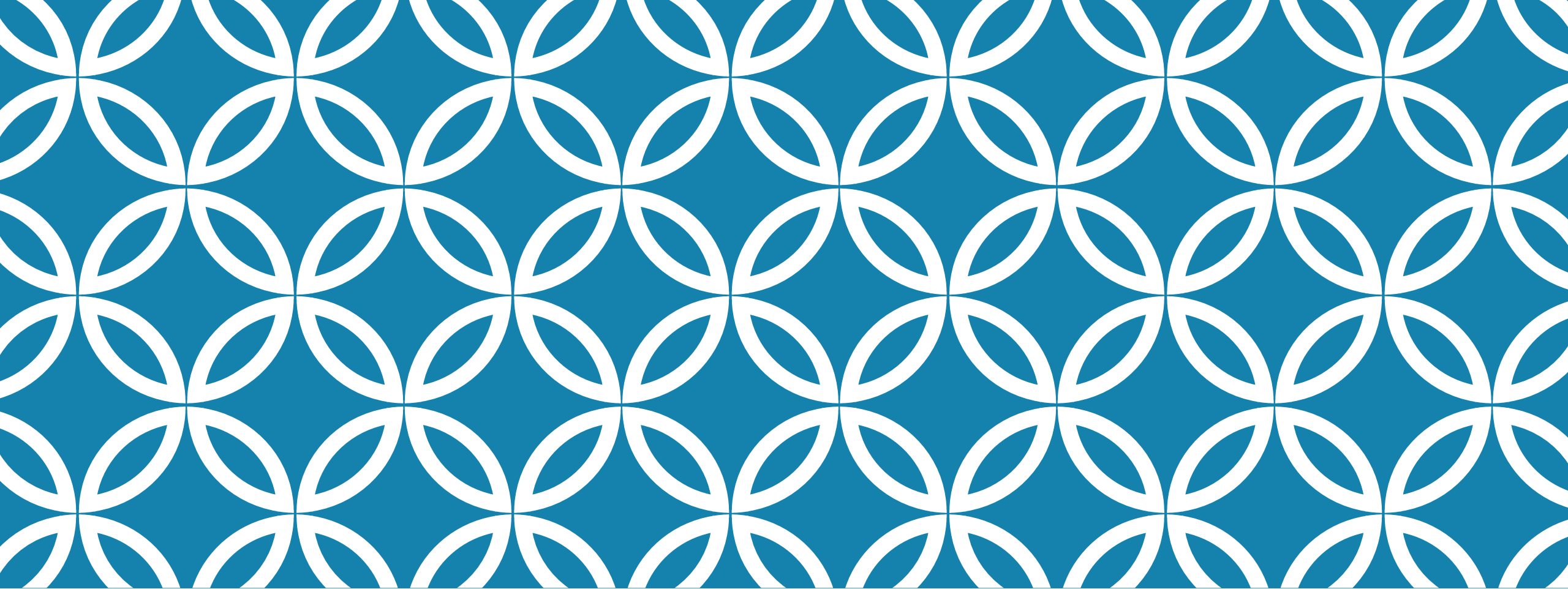




STATIC PROGRAM ANALYSIS

Lecture 17 – Escape Analysis

COMPILER OPTIMIZATION ON OBJECTS

Can a given object escape a method?

- Is this memory location can be reached outside of this stack frame
- Yes: should allocate on a heap
- No: can allocate on the stack

Stack allocation vs. heap allocation

- Faster to allocate -- no need to synchronize allocator with other threads
- No garbage collection

Definition:

- Method-escape: an object O escape method M if the lifetime of O exceeds the lifetime of M

ABSTRACTION

<https://dl.acm.org/doi/10.1145/320384.320386>

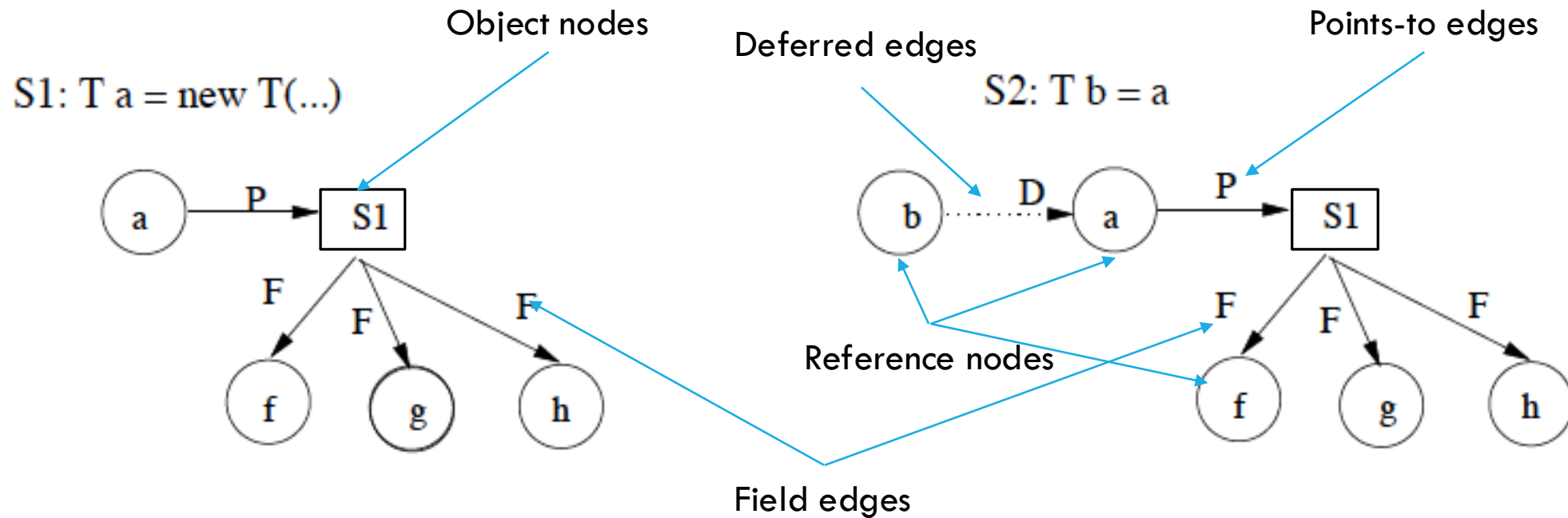
Connected graph

- Establishes reachability relations between objects and object references.
- Summarizes the affect of the method and can be used in different calling contexts.

A directed graph $CG = (N_o \cup N_r \cup N_f \cup N_g, E_r \cup E_d \cup E_f)$

- N_o is a set of objects.
- N_r is a set of reference objects.
- N_f is a set of non-static field nodes.
- N_g is a set of static field nodes, i.e., all global variables in the program.
- E_r is a set of points-to edges. If $x \rightarrow y \in E_r$ then $x \in (N_g \cup N_r \cup N_f)$ and $y \in N_o$
- E_d is a set of deferred edges. If $x \rightarrow y \in E_d$ then $x, y \in (N_g \cup N_r \cup N_f)$
- E_f is a set of field edges. If $x \rightarrow y \in E_f$ then $x \in N_o, y \in (N_g \cup N_f)$

EXAMPLE OF CONNECTED GRAPH



Deferred edges model assignments that just copy the reference from one variable not another.

Fields of the same object type have the same ids, i.e., $fid(O1.f) = fid(O2.f)$

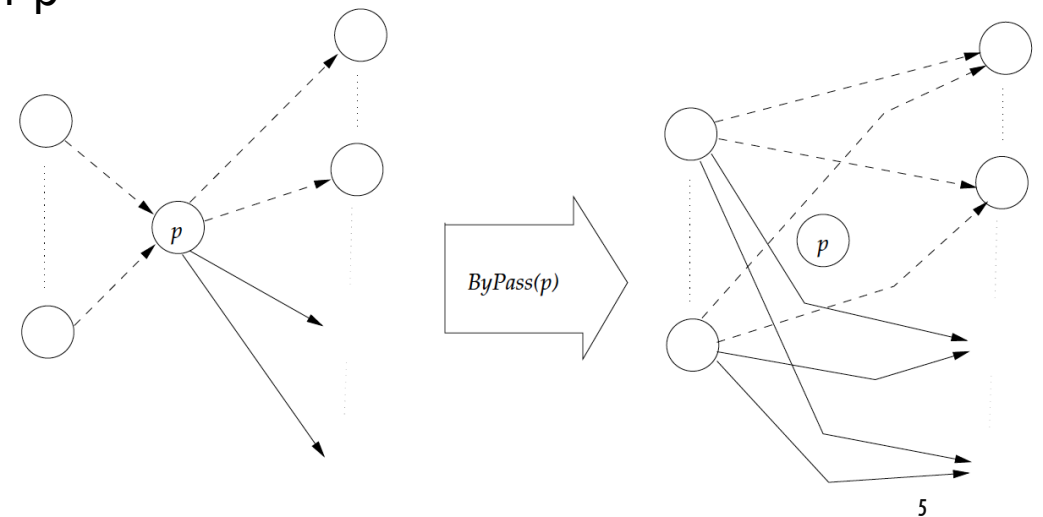
To determine a points-to set of a reference node: traverse deferred edges until P edge is found, explore all paths.₄

BASIC IDEA

let CG be a connection graph for a method M , and let O be an object node in CG. If O can be reached in CG from any node that escapes M then O escapes.

Construct an intraprocedural CG

- Simplification of statement $a.b.c = \text{new } T() \rightarrow t = \text{new } T(); t1 = a.b; t2.c = t;$
- $\text{ByPass}(p)$ redirects the incoming deferred edge of p to the successor nodes of p , saving the type of the outgoing edges of p , removes incoming/outgoing edges of p



COMPUTATION

Standard dataflow on CG

For a statement S

- Incoming: $C_i^S = \sqcup_{\{r \in \text{Pred}(s)\}} C_o^r$
- Outgoing: $C_o^i = f^s(C_i^S)$
- $\sqcup \equiv$ union of all nodes and edges of graphs

TRANSFER FUNCTION

Statement type

- $p = new \tau()$: create a new object node O (if one does not exist for his site). Flow-sensitive: apply $ByPass(p)$ and then add new points to edge from p to O . Flow-insensitive: just add the points-to edge from p to O
- $p = q$: flow sensitive, apply $ByPass(p)$ and then add a deferred edge from p to q . Flow-insensitive: just add the deferred edge.
- $p.f = q$ Let $U = PointsTo(p)$
 - If empty assume that p goes from outside of this method, create a phantom object node O_{ph} . During inter-procedural analysis they are replaced by actual objects.
 - Now U is not empty. Then just get the field node to which each $u \in U$ is pointing to nodes with the same field id (or create ones), then crate deferred edges between those field and q .
 - Cannot kill $p.f$ ($ByPass(p.f)$) anymore
- $p = q.f$. Let U be the set of all nodes q points to (through deferred edges), and V the set of nodes elements of U pointing to fields with the same field ids.
 - U is empty then created a phantom node, V is empty create a field node.
 - Flow-sensitive: apply $ByPass(p)$ and then add deferred edges from p to all elements in V , flow insensitive: just add edges.

EXAMPLE

S1: T1 a = new T1(...);

S2: T1 b = a;

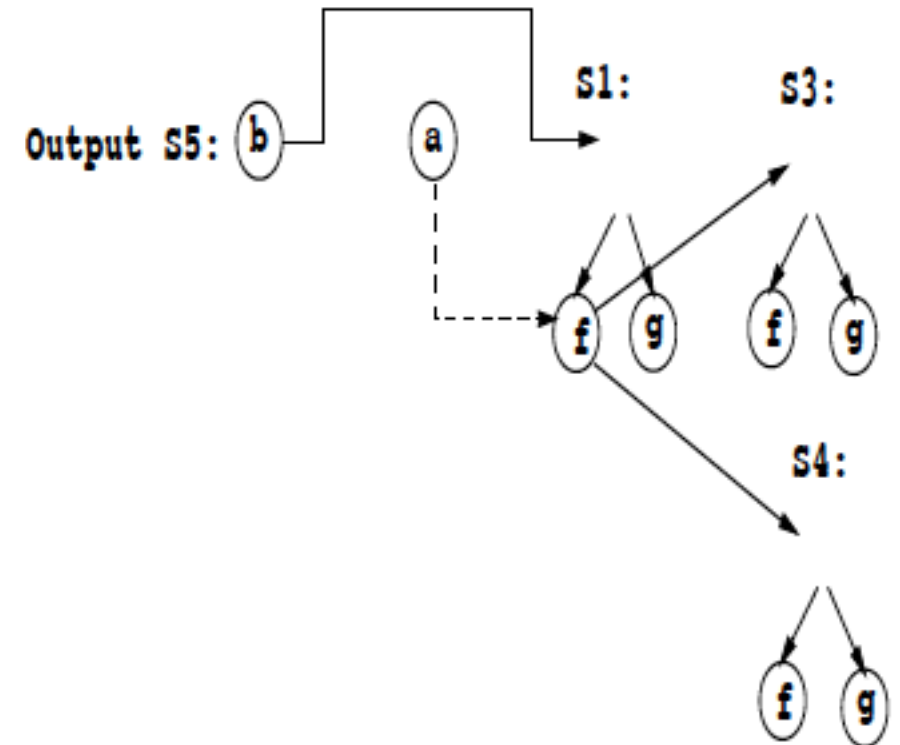
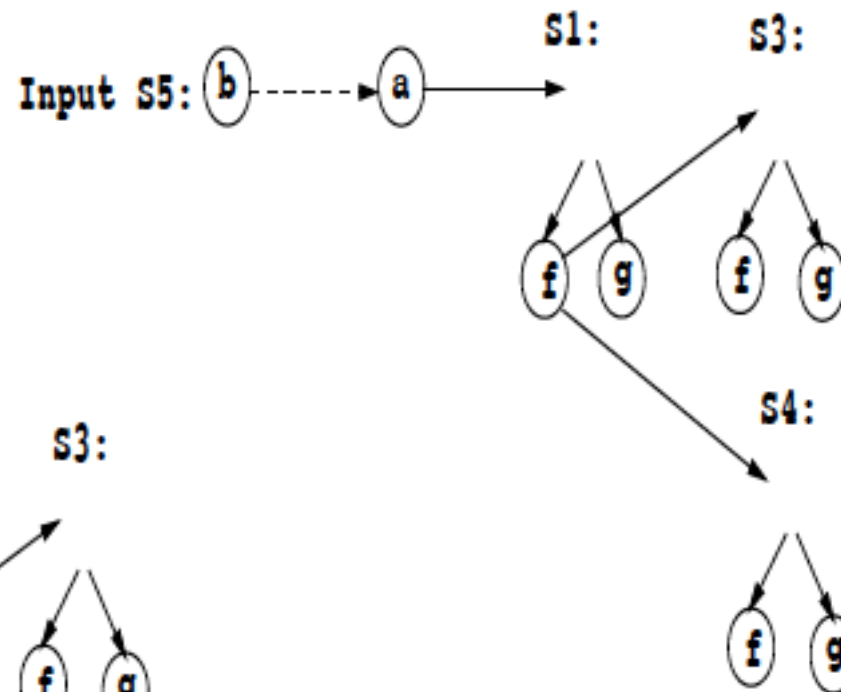
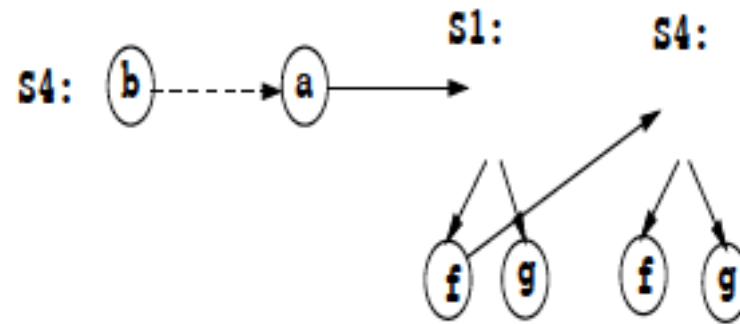
if()

S3: a.f = new T1(...);

else

S4: b.f = new T1(...);

S5: a = b.f;



REACHABILITY ANALYSIS

Traverse the graph and propagate escaping info

Intraprocedural:

Mark all phantom nodes (e.g., argument), global nodes as escaping, add to Q

Mark remaining nodes as non-escaping

For each $m \in Q$

For each outgoing edge $m \rightarrow n$

If n is not marked as escaped, then mark as one and add to Q

INTERPROCEDURAL ANALYSIS

Connect graph at method entry point

- Process $f_i = a_i$ for each formal parameter f_i and its corresponding phantom reference node a_i

Connect graph at method exit

- $return = ref$ for each returned reference
- ByPass each reference node

Apply reachability analysis on M

Find escaping nodes (globally and argument escaping)

Partition CG of M into e parts:

- Global: set of reachable from a GlobalEscape node
- Arg – set of reachable from an ArgEscape node
- Local – set of noEscape nodes.

Connect graph before and after a call site

Propagate effects of the callee to caller

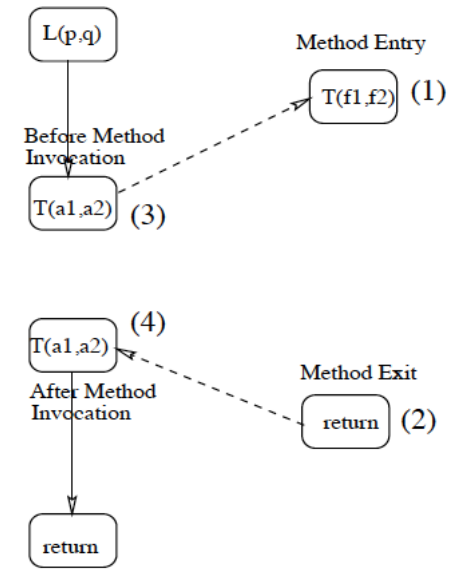
CG PRACTICE

```
class ListElement
{
    int data;
    ListElement next;
    static ListElement g = null;
    ListElement() {data = 0; next = null;}

    static void L(int p, int q)
    {
S0:    ListElement u = new ListElement();
        ListElement t = u;
        while(p > q)
        {
S1:        t.next = new ListElement();
            t.data = q++;
            t = t.next;
        }
S2:    ListElement v = new ListElement();
        NewListElement.T(u, v);
    }
}
```

```
class NewListElement
{
    ListElement org;
    NewListElement next;
    NewListElement() {org = null; next = null;}

    static void T(ListElement f1, ListElement f2)
    {
S3:    NewListElement r = new NewListElement();
        while(f1 != null)
        {
S4:        r.org = f1.next;
S5:        r.next = new NewListElement();
            . . . // do some computation using r
            . . . // w/o changing the data structure
S6:        r = r.next;
            if(f1.data == 0)
            {
S7:        ListElement.g = f2;
            }
            f1 = f1.next;
        }
    }
}
```



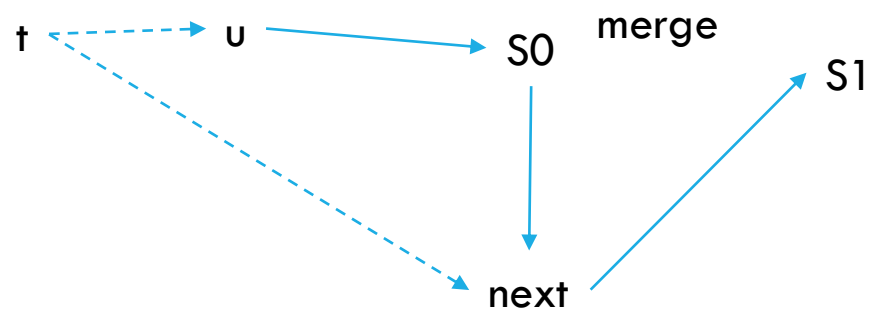
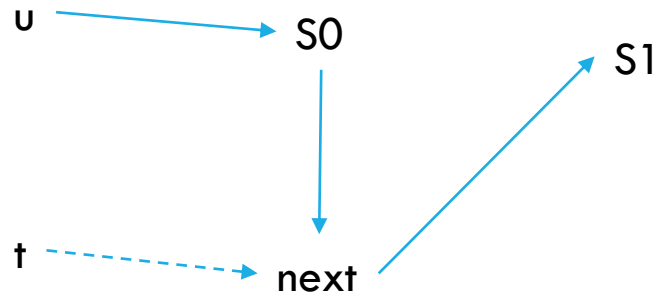
```
ListElement u = new ListElement();
```



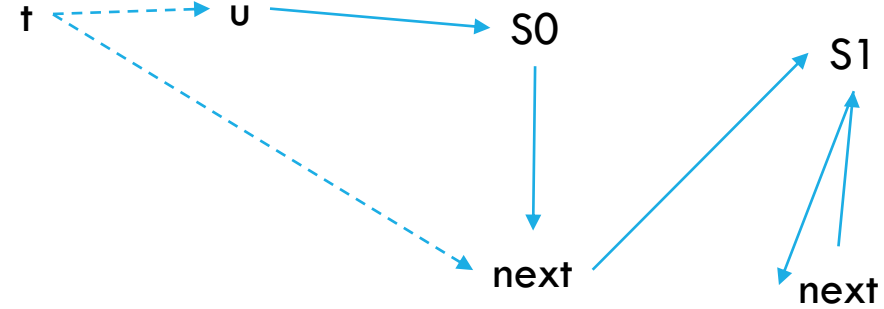
```
ListElement t = u;
```



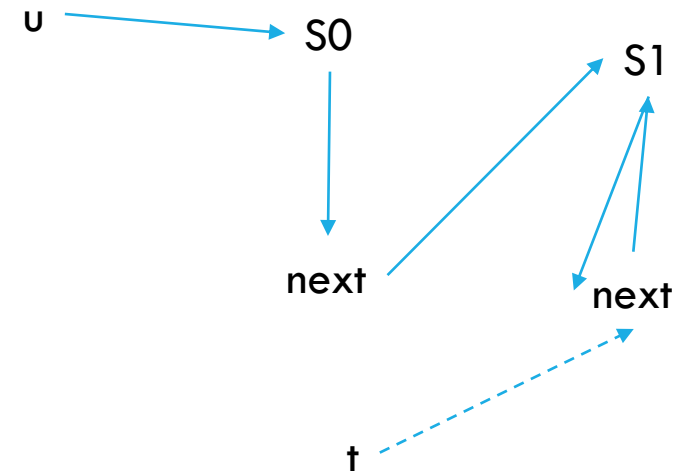
```
t.next = new ListElement();  
t = t.next;
```



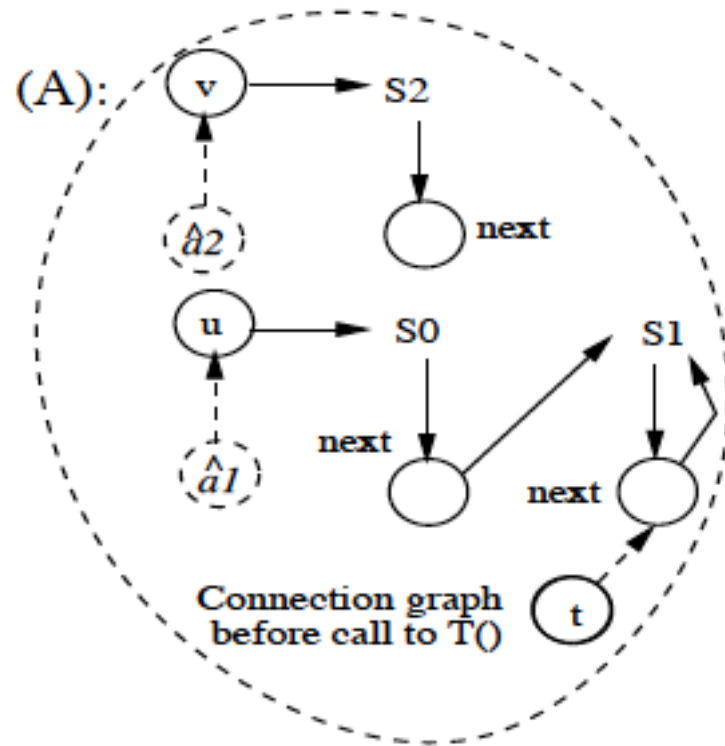
```
t.next = new ListElement();
```



```
t = t.next;
```



Then on union of the flows, we get the graph on the next page with red directed edges



Mine

